

CSCE 647 IMAGE SYNTHESIS · MODULE 12 · LECTURE NOTES

Denoising and Adaptive Sampling

Spending samples where they help, and cleaning up what is left.

MODULE OUTCOMES

- Estimate per-pixel variance and drive adaptive sampling with it.
- Explain how auxiliary feature buffers make denoising possible.
- Implement an edge-aware à-trous filter.
- Compare hand-designed and learned denoisers honestly.
- Explain ReSTIR's core idea and why it mattered.

1 Adaptive sampling

VARIANCE IS WILDLY UNEVEN ACROSS AN IMAGE

A flat wall lit directly by a large area light converges in a handful of samples. A caustic on the floor beside it, or a glossy reflection of a small bright light, may need thousands to reach the same error.

Uniform sampling gives both the same budget, which simultaneously wastes effort on the easy regions and starves the hard ones.

The fix is to estimate per-pixel variance and allocate accordingly. The estimate is essentially free: accumulate the sum and the sum of squares of the samples already drawn, and the running variance follows.

Typical gains are two to four times at equal quality. Worth having, and not transformative on its own — the techniques later in this module matter more.

ADAPTIVE SAMPLING INTRODUCES A SMALL BIAS

If the decision to draw more samples depends on the values of the samples already drawn, those samples are **no longer independent of the stopping rule**, and the estimator is not strictly unbiased.

The mechanism is concrete: a pixel whose first few samples happen to be low looks converged, gets starved of further samples, and stays low. The effect is systematic and slightly darkens regions with high-variance, high-value transport.

The standard defences are to draw a fixed initial budget before any adaptation, and to cap the ratio between the largest and smallest per-pixel allocation.

In practice the residual bias is small and essentially everyone accepts it. The point is to accept it knowingly — do not describe a renderer as unbiased while it adapts.

2 Feature buffers

RENDERING IS A UNIQUELY FAVOURABLE CASE FOR DENOISING

Denoising a photograph is fundamentally underdetermined: the algorithm must infer which variations are signal and which are sensor noise, with no information beyond the image itself.

A renderer knows a great deal that a camera does not. It can output, with no noise at all, the surface albedo, the shading normal, the depth, the object identifier, and the motion vector for every pixel — because these are properties of the first intersection, which is deterministic.

These buffers tell the filter precisely where the edges are. It can then blur aggressively along a smooth surface while refusing to blur across a geometric boundary, a depth discontinuity, or an object edge.

This is why render denoising works far better than photo denoising. The genuinely hard part of the problem — identifying structure — was handed to us by the renderer.

Buffer	What it provides	Cost
Albedo	The surface colour at the first hit, so texture detail is not mistaken for noise and smoothed away.	Free — it is the material evaluated at the primary hit.
Normal	Geometric edges, creases, and curvature.	Free.
Depth	Depth discontinuities; also enables reprojection.	Free.
Object ID	Hard boundaries between distinct objects that may happen to have similar normals and depths.	Free.
Motion vectors	Where each pixel's surface was in the previous frame, enabling temporal accumulation.	Free in an animation; requires the previous frame's transforms.
Variance	How much to trust each pixel.	Estimated from the samples (\$1).

DEMODULATE BY ALBEDO BEFORE FILTERING

Filtering the final colour blurs texture along with noise: a noisy brick wall comes back as a smooth brown wall.

Instead, divide by the albedo buffer before filtering and multiply it back afterwards. The filter then operates on the *illumination* alone, which is smooth and low-frequency, while the texture — which was never noisy — is reapplied intact.

This single step is the largest quality improvement available for a modest amount of code, and it is frequently omitted in first implementations.

Handle the edge cases: guard against division by a near-zero albedo, and keep emissive surfaces out of the demodulation, since their radiance is not a product of albedo and illumination.

```

vec3 atrous(const Buffers& b, ivec2 p, int step) {
    vec3 sum(0); float wsum = 0;
    for (int dy = -2; dy <= 2; dy++)
        for (int dx = -2; dx <= 2; dx++) {
            ivec2 q = p + ivec2(dx, dy) * step;          // the "hole"
            if (!inside(q)) continue;
            float wn = powf(fmaxf(0.f, dot(b.normal[p], b.normal[q])), SIGMA_N);
            float wz = expf(-fabs(b.depth[p] - b.depth[q]) / (SIGMA_Z + 1e-6f));
            float wl = expf(-fabs(lum(b.color[p]) - lum(b.color[q]))
                               / (SIGMA_L * sqrtf(b.variance[p]) + 1e-6f));
            float w = kernel[dy+2][dx+2] * wn * wz * wl;
            sum += b.color[q] * w; wsum += w;
        }
    return wsum > 0 ? sum / wsum : b.color[p];
}

```

The **à-trous** construction runs several passes with an increasing gap between the sampled taps, achieving the effect of a very wide filter at linear rather than quadratic cost. Each of the three edge-stopping weights suppresses blurring across a different kind of discontinuity.

THE VARIANCE-SCALED LUMINANCE WEIGHT IS THE CLEVER PART

The luminance term w_l suppresses blurring between pixels whose brightness differs substantially — which protects genuine edges in the illumination, such as a shadow boundary.

But in a noisy image, neighbouring pixels differ substantially because of noise, not because of an edge. A naive luminance weight therefore refuses to blur exactly where blurring is most needed, and preserves the noise as though it were detail.

Scaling the tolerance by the estimated standard deviation fixes this. Where variance is high, large differences are interpreted as noise and blurred through; where variance is low, the same difference is interpreted as a real edge and preserved.

This is the central idea of SVGF (spatiotemporal variance-guided filtering), and it is what makes the filter work at low sample counts rather than merely at moderate ones.

3 Learned denoisers

	Hand-designed (à-trous, SVGF)	Learned (OIDN, OptiX)
How it works	Explicit edge-stopping weights, designed and tuned by hand.	A convolutional network trained on pairs of noisy and converged renders.
Quality at 4–16 spp	Adequate; visible over-blurring.	Substantially better. This is where the gap is largest.
Failure mode	Predictable: over-blurring, ghosting on motion, loss of fine shadows.	Unpredictable: may smooth away real detail, or hallucinate detail that was never in the scene because it resembles the training data.
Tunability	Parameters you can reason about and adjust.	Retrain, or accept the result.
Cost	Fast; viable in real time.	Heavier; needs a GPU. Fine for offline, and increasingly viable interactively.
Status	Used in real time and as a component of larger systems.	The production default for offline rendering.

DENOISING IS BIASED, AND USING IT IS STILL CORRECT

A denoiser modifies each pixel using information from its neighbours. **The output is not an unbiased estimate of the rendering equation**, and no amount of care makes it one.

It can remove genuine high-frequency content — fine texture detail, small sharp highlights, thin contact shadows — and a learned denoiser can introduce detail that has no counterpart in the scene, because the network has learned what such regions usually look like.

And using one is still the right engineering decision. At a fixed compute budget, a denoised 64-sample image is usually closer to the converged truth, by any error metric, than a noisy image at the sample count the same budget would otherwise buy.

What is not acceptable is silence about it. Report that an image is denoised, give the sample count, name the denoiser, and show the noisy input and a converged reference beside the result. Project 2 requires exactly this, and the requirement is not pedantry — a denoised image presented as a render is a claim about convergence that has not been earned.

4 ReSTIR

Reservoir-based spatiotemporal importance resampling, from Bitterli et al. in 2020. It is the most consequential rendering result of the last decade, and the core idea is simple.

- 1 **Reservoir sampling** maintains a single sample selected from an arbitrarily long stream, with the correct probability, in constant memory. The technique itself is decades old.

- 2 **Resampled importance sampling** draws M candidate samples from a cheap distribution and resamples one of them according to a better target distribution — giving approximately the better distribution's quality for roughly the cheap one's cost.
- 3 **Spatial reuse:** a neighbouring pixel usually shades a similar surface under similar lighting, so *its* chosen light sample is probably good here too. Combine the reservoirs.
- 4 **Temporal reuse:** the same surface point last frame is probably still well served by the sample chosen then. Combine across frames using motion vectors.

WHY IT MATTERED

It attacks the constant in $1/\sqrt{N}$, which Module 05 identified as the only available target — and it attacks it by orders of magnitude rather than by factors, because each pixel effectively benefits from work done by all its neighbours across many frames.

It changed what is possible rather than merely what is fast. Direct illumination from millions of light sources, which Module 09 described as the many-lights problem, went from intractable to real time at one sample per pixel.

The costs are genuine. Reused samples are correlated, so the residual error is structured rather than random — which, per §3, is exactly the kind of error denoisers handle worst. And the bias correction, which requires MIS weights across reservoirs, is subtle; early implementations were biased in ways that took time to diagnose.

It generalises. ReSTIR GI extends the idea to indirect illumination and ReSTIR PT to full path reuse, which is an active area and the most likely place for the next significant result.

DENOISING CHANGED WHAT THE FIELD OPTIMISES FOR

Before practical denoising, variance reduction was the only lever available, and the sophisticated algorithms of Module 11 competed directly on it.

The question now is different: what kind of noise does the denoiser handle well? That is a different objective, and it favours different methods.

High-frequency, spatially uncorrelated noise denoises beautifully. Fireflies, correlated splotches, and structured error do not. So an algorithm with somewhat higher variance but better-behaved noise can now beat one with lower variance outright.

This is why blue-noise sampling (Module 06) matters despite not improving RMSE, and it is why Metropolis's splotchy error is a serious liability rather than a cosmetic complaint. The metric changed, and so did the ranking.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Schied et al. — Spatiotemporal Variance-Guided Filtering (SVGF, free)	The filter of §2, including the variance estimation and temporal accumulation. The paper to implement from.
Intel Open Image Denoise	A free, production-quality learned denoiser with a simple API. Use it as the comparison point for your own filter.
Bitterli et al. — Spatiotemporal Reservoir Resampling (ReSTIR, free)	The §4 paper. Read the supplemental material for the bias discussion.
Wyman & Panteleev › A Gentle Introduction to ReSTIR (SIGGRAPH course, free)	Far more approachable than the paper. Start here.
Zwicker et al. — Recent Advances in Adaptive Sampling and Reconstruction (survey, free)	The survey covering §1 and §2, including the bias analysis of adaptive sampling.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement per-pixel variance estimation and output it as an image. The variance map is informative on its own — it shows exactly which transport is expensive.
- 2 Implement adaptive sampling driven by that estimate. Report equal-time RMSE against uniform sampling.
- 3 Demonstrate adaptive sampling bias: use no initial fixed budget and an uncapped allocation ratio, and compare the converged result against a uniform render.
- 4 Output albedo, normal, depth, and object ID buffers. Confirm they are noise-free.
- 5 Implement the à-trous filter. Then disable each edge-stopping weight in turn and document what breaks in each case.
- 6 Implement albedo demodulation. Compare a filtered brick wall with and without it — this comparison belongs in your portfolio.
- 7 Replace the fixed luminance tolerance with the variance-scaled version and report the difference at 4 and 64 samples per pixel.
- 8 Run Intel Open Image Denoise on the same inputs and compare against your filter at 4, 16, and 64 samples per pixel. Report RMSE and look at the images — the ranking may differ.
- 9 Find a case where the learned denoiser removes real detail or invents it. Document it honestly; such cases are not hard to construct.
- 10 Implement reservoir sampling for light selection with spatial reuse only. Report the variance reduction on a scene with 1,000 lights.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does adaptive sampling help, and what gain is typical?
- 2 How does adaptive sampling introduce bias, and what are the standard defences?
- 3 Why is denoising a render easier than denoising a photograph?
- 4 Name five noise-free buffers and say what each contributes.
- 5 What is albedo demodulation, why does it matter, and what edge cases need handling?
- 6 What does the à-trous construction achieve, and what do the three edge-stopping weights do?
- 7 Why must the luminance weight be scaled by variance?
- 8 Compare hand-designed and learned denoisers on quality, failure mode, and tunability.
- 9 In what sense is denoising biased, and what must you report?
- 10 What is ReSTIR's core idea, what did it change, and what are its costs?